06. Mechanism: Limited Direct Execution

How to efficiently virtualize the CPU with control?

OS는 physical CPU를 **time sharing**를 통해 공유해야함

Issue

Performance (성능)

시스템에 과도한 overhead(부담)을 주지 않으면서 어떻게 virtualization을 구현할까?

Control (제어 문제)

CPU에 대한 제어권(control)을 유지하면서 어떻게 프로세스를 효율적으로 실행할 수 있을까?

- 제어권이 없다면 프로세스가 영원히 실행되어 시스템을 장악하거나 접근하면 안되는 정보에 접근할 수 있다.

Direct Execution

- 위에서 언급된 Process execution을 수행하는 하나의 방법
- 그냥 CPU 위에서 Direct하게 돌려! - Program 실행 중간에 별다른 처리 없이
    - OS는 Program에 대한 초기, 후기 작업만을 담당

OS	Program
1. Process List의 entry를 생성	
2. Program에 대한 Memory 할당	
3. Program을 Memory에 로드	

4. Stack Set-up (argc와 argv로)	
5. 레지스터 초기화	
6. Execute call main()	
	7. main() 실행
	8. main() return
9. Process의 memory free	
10. process list에서 제거	

> ⭐ **Running programs에 limit을 걸지 않는다면**
> **OS는 제어권이 없는 것이고 그냥 Library로서 동작한다고 볼 수 있음**

띠용용?

3번 단계) Program을 Memory에 로드

- 실행 가능한 **프로그램의 Code(코드)와 Data(전역 데이터)**를 디스크에서 RAM(메인 메모리)으로 불러옴
- 대상: 실행 파일(.exe, .out 등)에 들어 있는 명령어(기계어 코드)와 전역 변수, 정적 변수 등의 데이터
- 목적: CPU가 프로그램을 실행하려면, 디스크에 있는 코드를 직접 읽을 수 없기 때문에 RAM에 올려두어야 함
- 프로그램 자체를 메모리에 준비하는 과정

4번 단계) Stack Set-up

- 프로그램이 실행될 때 사용할 스택 영역을 메모리에 할당하고 초기화하는 작업
  - **Stack(스택)**은 프로그램 코드와는 별개의 영역
- 대상: 지역 변수, 함수의 매개변수, 함수의 반환 주소 등을 저장할 빈 메모리 공간
- 목적: 함수 호출과 반환 시 필요한 데이터를 임시로 저장하기 위한 공간을 마련
- 특히, 앞서 설명한 main() 함수의 argc와 argv 인자도 이 스택에 초기화
- 프로그램이 실행될 때 사용할 작업 공간을 준비하는 과정

> 요약
> 3번: 책(프로그램 코드)을 도서관(메모리)에 가져다 놓는 것

> 4번: 그 책을 읽고 필기하며 계산할 책상(스택)을 준비하는 것

Recap: Process Creation

1. Load

- OS는 Program **code**를 메모리에 Load → process의 address space로
- Program은 원래 disk에 **executable format**으로 저장되어 있음 ex) a.out
- 이때 OS는 **lazy**한 방식으로 process를 load함

> **Lazy Loading**
> code와 data 중 program execution에 필수적인 일부분만 Loading하는 것

2. Run-Time Stack Set-Up

- local variables, function parameters, return address등
- main함수의 Parameter인 **argv 배열, argc도 초기화**
- **Register 초기화도** 이 단계에서 진행 - 인터넷에서 찾은 거

3. Heap Creation

- Explicit Dynamic Memory Allocation시 필요한 heap
  - ex) malloc(), free()

4. 추가적인 initialiation tasks

- **Input/output (I/O) setup**
  - Process의 기본 open file descriptors 3개: STDIN, STDOUT, STDERR

> **1, 2, 3 ,4는 OS가 하는 것이라고 보면 됨, 이후 process에게 CPU 제어권 넘겨주는 것**

5. Start the program

- main()에서 시작

- CPU의 제어권을 새로 만들어진 **process에게 control 넘겨줌**

Issues

1. User can do wrong thing

```
int *i;
i = 0 ;
*i = 1 ;
```

2. Getting the control back from CPU is not easy

```
i = -1;
while (i < 0)
    do something;
```

Problem 1 : Process가 Restricted Operations를 수행하고자 할 때 문제가 발생

- Disk에 대한 I/O Request
- CPU / Memory와 같은 System Resource에 대한 접근 요청

Solution 1: Grant everything

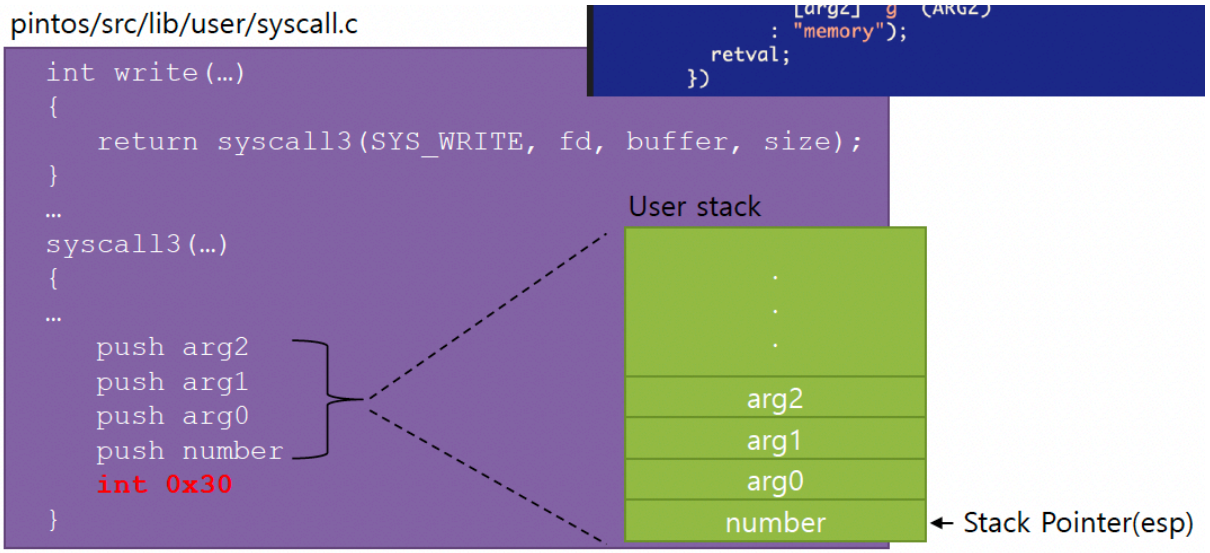
Solution 2: Use protected control transfer

- **User mode**: Applications은 hardware resources에 대한 full access 불가능
- **Kernel mode**: The OS has access to the full resources of the machine
  - 인터넷에서 찾은거
    - kernel mode는 또 supervisor mode, system mode, privileged mode로 구분
    - 특정 instruction은 privileged일 때만 사용 가능 ex) I/O Request, File System Request

- 즉, User Mode에 있는 Process는 Non-privileged Operation만 수행 가능
  - 그러나!!! User Mode의 Program도 Privileged(Restrictive) Operation이 필요함
  - 따라서 User는 System Call과 같은 Interface를 통해 Kernel Mode 전환

System Call

- Accessing the file system, Creating and destroying processes, Communicating with other processes, Allocating more memory...
- User Mode를 Kernel Mode로 전환, Return할 때, User로 돌아감
  - system call 함수는 내부적으로 Trap (Interrupt)



```
#define syscall3(NUMBER, ARG0, ARG1, ARG2)
        ({
          int retval;
          asm volatile
            ("pushl %[arg2]; pushl %[arg1]; pushl %[arg0]; "
             "pushl %[number]; int $0x30; addl $16, %%esp"
               : "=a" (retval)
               : [number] "i" (NUMBER),
                 [arg0] "g" (ARG0),
                 [arg1] "g" (ARG1),
                 [arg2] "g" (ARG2)
               : "memory");
          retval;
        })
```

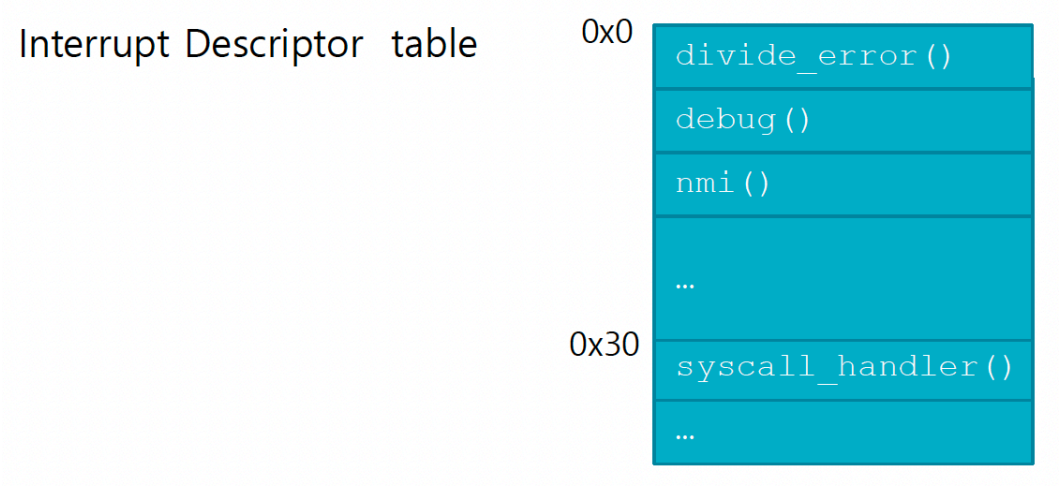
arg2, arg1, arg0, syscall number 넣고, interrupt 명령어 걸고 다시 복귀하면 스택 포인터 증가시키기

Register systemcall handler at IDT

Interrupt descriptor table

- Table of the locations of the interrupt handlers for the associated interrupt number.
- Register the systemcall handler at interrupt 0x30(src/userprog/syscall.c).

```
void
syscall_init (void)
{
  intr_register_int (0x30, 3, INTR_ON, syscall_handler, "syscall");
}
```



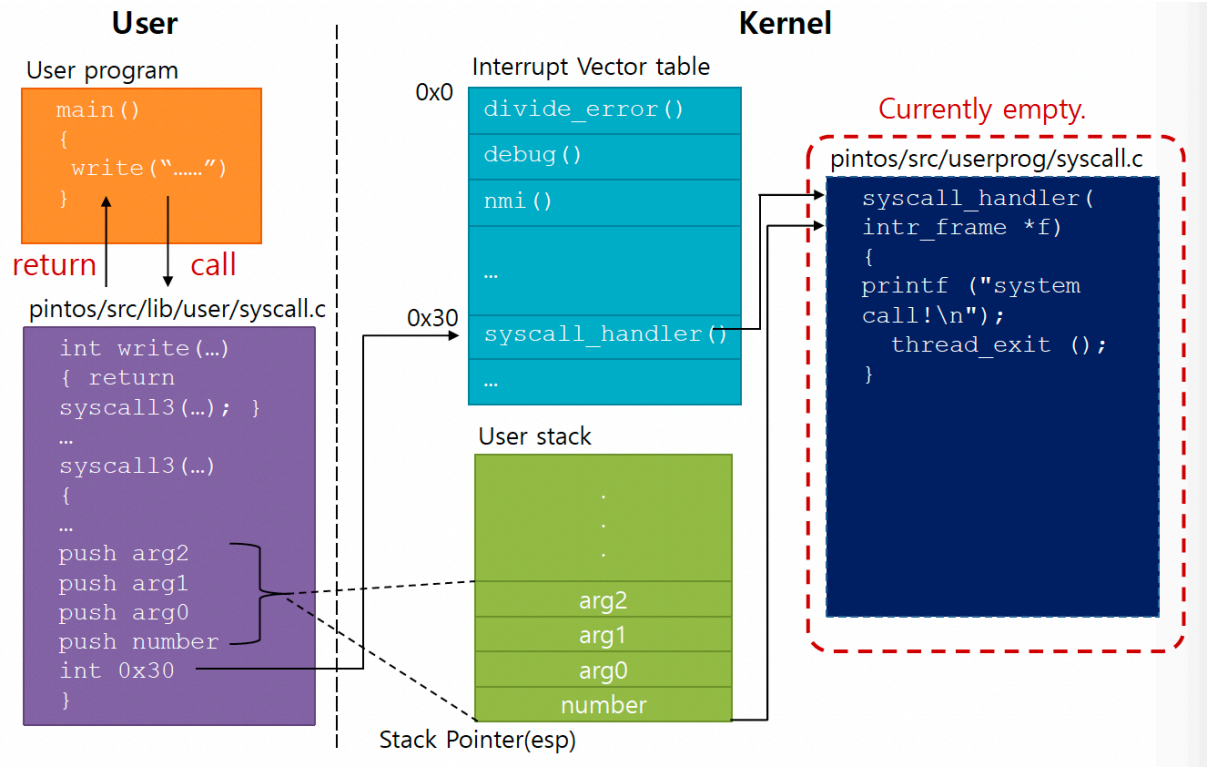
int $0x30 실행 과정:

1. CPU가 IDT 배열에서 `idt[0x30]` 읽음
2. 거기 저장된 **주소**(0xC0105A00)를 가져옴
3. 그 주소로 PC 점프
4. 그때부터 **실제 명령어** 실행

정리:

- **IDT = 주소 배열** (데이터)
- **핸들러 = 실제 명령어** (코드)
- IDT는 핸들러를 **가리키는** 포인터들의 모음!

Call flow(Pintos): write()



kernel stack이랑 process-struct 차이점

cache는 context switch에 포함안된다

Key Concept of System Call

Trap instruction

```
int n
```

- Raise the privilege level to kernel mode
- Save registers at the kernel stack. (eip, cs, eflags, esp, ss)
- Locates the destination in the kernel - 이전에 말한 IDT?
- Jumps to the destination.

- Interrupt number for system call
  - 0x64 @xv6 | 0x30 @Pintos | e.g. int 0x64

Return-from-Trap instruction

- Return into the calling user program
- Reduce the privilege level back to user mode

What code to run in trap?

Sources of trap

- Completion of disk IO
- Keyboard interrupt
- System call

Trap handler

- the code to run for each interrupt number (trap number)

Trap table

- The address of the trap handlers.
- The OS informs the hardware of the locations of trap handlers.
- The instruction to inform the location of the trap table is also privileged instruction.
- You cannot execute this instruction in user mode.

Limited Direction Execution

OS @ boot(kernel mode)	**Hardware**	
initialize trap table		

	remember address of ... syscall handler	
OS @ run(kernel mode)	**Hardware**	**Program (user mode)**
Create entry for process list Allocate memory for program Load program into memory Setup user stack with argv Fill kernel stack with reg / PC **return-from -trap**		
	restore regs from kernel stack move to user mode jump to main	
		Run main() ... Call system **trap** into OS
	save regs to kernel stack move to kernel mode jump to trap handler	
Handle trap Do work of syscall **return-from-trap**		
	restore regs from kernel stack move to user mode jump to PC after trap	
		return from main trap (via exit())
Free memory of process Remove from process list		

Problem 2: Switching Between Processes

- How can the OS regain control of the CPU so that it can switch between processes?
  - A cooperative Approach: **Wait for system calls**
  - A Non-Cooperative Approach: **The OS takes control**

Solution 1: A cooperative Approach: Wait for system calls

- Processes **periodically give up the CPU** by making **system calls** such as yield.
  - The OS decides to run some other task.
  - Application also transfer control to the OS when they do something illegal.
    - Divide by zero
    - Try to access memory that it shouldn't be able to access
- Ex) Early versions of the Macintosh OS, The old Xerox Alto system

> **A process gets stuck in an infinite loop. → Reboot the machine**

Solution 2: A Non-Cooperative Approach: OS Takes Control

A timer interrupt

- During the boot sequence, the OS start the timer.
- The timer raise an interrupt every few milliseconds.
- When the interrupt is raised
  - The currently running process is suspended.
  - Save enough of the state of the process.
  - A pre-configured interrupt handler in the OS runs

> **A timer interrupt gives OS the ability to run again on a CPU.**

Saving and Restoring Context

- Scheduler makes a decision
  - Whether to continue running the **current process**, or switch to a **different one**.

- If the decision is made to switch, the OS executes context switch

Context Switch

- A low-level piece of assembly code
  - **Save a few register values** for the current process onto its kernel stack
    - General purpose registers
    - PC
    - kernel stack pointer
  - **Restore a few** for the soon-to-be-executing process from its kernel stack
  - **Switch to the kernel stack** for the soon-to-be-executing process

Limited Direct Execution Protocol (Timer interrupt)

OS @ boot **(kernel mode)**	**Hardware**	
initialize trap table		
	remember address of ... syscall handler timer handler	
start interrupt timer		
	start timer interrupt CPU in X ms	
OS @ run **(kernel mode)**	**Hardware**	**Program (user mode)**
		Process A ...
	timer interrupt save regs(A) to k-stack(A) move to kernel mode jump to trap handler	
	(Cont.)	

Handle the trap Call switch() routine save regs(A) to proc-struct(A) restore regs(B) from proc-struct(B) switch to k-stack(B) **return-from-trap (into B)**		
	restore regs(B) from k-stack(B) move to user mode jump to B's PC	
		Process B ...

```
1  # void swtch(struct context **old, struct context *new);
2  #
3  # Save current register context in old
4  # and then load register context from new.
5  .globl swtch
6  swtch:
7          # Save old registers
8          movl 4(%esp), %eax          # put old ptr into eax
9          popl 0(%eax)                # save the old IP
10         movl %esp, 4(%eax)          # and stack
11         movl %ebx, 8(%eax)          # and other registers
12         movl %ecx, 12(%eax)
13         movl %edx, 16(%eax)
14         movl %esi, 20(%eax)
15         movl %edi, 24(%eax)
16         movl %ebp, 28(%eax)
17
18         # Load new registers
19         movl 4(%esp), %eax          # put new ptr into eax
20         movl 28(%eax), %ebp         # restore other registers
21         movl 24(%eax), %edi
22         movl 20(%eax), %esi
23         movl 16(%eax), %edx
24         movl 12(%eax), %ecx
25         movl 8(%eax), %ebx
26         movl 4(%eax), %esp          # stack is switched here
27         pushl 0(%eax)               # return addr put in place
28         ret                         # finally return into new ctxt
```

Worried About Concurrency?

- What happens if, during interrupt or trap handling, another interrupt occurs?
- OS handles these situations
  - **Disable interrupts** during interrupt processing
  - Use a number of sophisticated **locking** schemes to protect concurrent access to internal data structures.